

A3 · Answer Key and Demo Script

CS425 · Interrogating the Transport Layer with Claude Code

INSTRUCTOR COPY

Graded pass/fail. Rehearsed on this Mac on Sep 27: the prompts on the page went to Claude Code word for word, and the transcripts below are its own `echo.py` and `burst.py` run with the page's commands. Ports and pids will differ; the shapes will not. Use port 5425: AirPlay Receiver holds *:5000 on macOS.

Say this before you hand out the sheet

"Chapter 3 makes three claims you will be tested on: UDP sockets are named by two things and TCP sockets by four, somebody is left holding TIME_WAIT, and TCP moves bytes, not messages. Today we watch each one happen. I drive Claude Code; you predict before I press enter, and you catch it when it is wrong. Last round, you drive." One minute, then paper.

BEFORE CLASS Logged in to Claude Code, font up, four terminal tabs. Rehearsal copies of `echo.py` and `burst.py` in a spare directory: if the wifi or Claude fails, switch to them and keep going.

ROUND 0 · MEET THE AGENT

- Trust.** Trusting a folder lets Claude read and change everything under it. An empty directory you just made holds nothing to lose; your home directory holds SSH keys, tokens and every other project.
- Approve once vs. always.** One approval covers one command you have read. "Yes, and don't ask again" lets it run that kind of command in this directory without asking, including runs you never see. Model approving one at a time, out loud.

ROUND 1 · DEMULTIPLEXING

Isuf -nP -i :5425, with two nc clients connected and two UDP datagrams sent									
Python	73383	shane	3u	IPv4	...	TCP	127.0.0.1:5425	(LISTEN)	
Python	73383	shane	4u	IPv4	...	UDP	127.0.0.1:5425		
Python	73383	shane	5u	IPv4	...	TCP	127.0.0.1:5425->127.0.0.1:62871	(ESTABLISHED)	
Python	73383	shane	6u	IPv4	...	TCP	127.0.0.1:5425->127.0.0.1:62872	(ESTABLISHED)	
nc	73387	shane	3u	IPv4	...	TCP	127.0.0.1:62871->127.0.0.1:5425	(ESTABLISHED)	
nc	73389	shane	3u	IPv4	...	TCP	127.0.0.1:62872->127.0.0.1:5425	(ESTABLISHED)	
POINT AT Both binds worked: TCP and UDP port numbers are separate namespaces. The server holds 1 LISTEN, 2 connected TCP sockets (one per four-tuple), and exactly 1 UDP socket no matter how many clients sent, because a UDP socket is named only by its own address and port. The server log shows a different source port for each datagram, all arriving on that one socket. A web server needs the four-tuple so thousands of clients can all talk to port 443 and still land in separate sockets.									

ROUND 2 · TIME_WAIT

netstat -an -p tcp grep 5425, just after the clients pressed Ctrl-C									
tcp4	0	0	127.0.0.1.5425	.*.*				LISTEN	
tcp4	0	0	127.0.0.1.62871	127.0.0.1.5425				TIME_WAIT	
tcp4	0	0	127.0.0.1.62872	127.0.0.1.5425				TIME_WAIT	
POINT AT The TIME_WAIT lines carry the <i>client's</i> ephemeral ports. Whoever closes first holds TIME_WAIT, for 2 MSL: about 30 s on macOS (<code>net.inet.tcp.msl</code> is 15000 ms; it measured 32 s), 60 s on Linux. It exists so a late duplicate segment from this connection cannot be mistaken for part of a new one on the same four-tuple, and so the final ACK can be resent if it was lost.									

- Server stopped first.** Now the server's side holds TIME_WAIT on port 5425. A restart without `SO_REUSEADDR` fails: [Errno 48] Address already in use (Linux: errno 98). In rehearsal Claude set `allow_reuse_address = True` unasked, so the restart worked, and a copy with it set to `False` failed; ask it why the line is there and hold its answer against this. If it left it out, the failure is the lesson.
- Grading the agent.** A common slip is "the server is left in TIME_WAIT" for a client-initiated close. Credit any group that caught it.

ROUND 3 · A BYTE STREAM IS NOT A MESSAGE

echo.py output while burst.py runs

```
TCP client 127.0.0.1:62960
TCP recv 11 bytes: b'onetwothree'
UDP client 127.0.0.1:62067
UDP recv 3 bytes: b'one'
UDP client 127.0.0.1:62067
UDP recv 3 bytes: b'two'
UDP client 127.0.0.1:62067
UDP recv 5 bytes: b'three'
```

POINT AT TCP: one recv, 11 bytes. If Claude's version also prints a 0 byte recv, that is the client's FIN; ask the room. The three sends became bytes in one stream and the receiver gets whatever has arrived; send boundaries are not preserved. UDP: three datagrams, 3, 3 and 5 bytes, one recv each. That is why P1 read until `\r\n`: the protocol, not TCP, marks where a reply ends. Without the one second sleep the TCP result varies run to run on loopback, which is why the prompt asks for it.

ROUND 4 · PART B

UDP checksum. $01010011 + 01100110 = 10111001$; $+ 01110100 = 1\ 00101101$; wrap the carry: sum 00101110 . Checksum is the complement, **11010001**. Watch for an agent answer that drops the wraparound carry.

Stop-and-wait. $L/R = 8000 / 10^9 = 8\ \mu\text{s}$. $U = 0.008 / 30.008 \approx 0.00027$ (0.027%). For 90%: $0.9 \times 30.008 / 0.008 \approx 3376$ packets in flight (3375 if L/R is dropped from the denominator; also fine).

Timeout. $\text{EstimatedRTT} = 0.875 \times 100 + 0.125 \times 120 = 102.5\ \text{ms}$. New estimate in DevRTT: $0.75 \times 5 + 0.25 \times 17.5 = 8.125$, timeout **135 ms**. Old estimate (RFC 6298's order): $0.75 \times 5 + 0.25 \times 20 = 8.75$, timeout **137.5 ms**. Both are right if the group can say which order; the credit is for noticing whether the agent says which. In rehearsal Claude used the old estimate (137.5 ms), cited RFC 6298, and offered 135 ms unprompted.

If you finish early. MSL is Maximum Segment Lifetime. `TIME_WAIT` is 2 MSL so a segment can die in one direction and its reply in the other. macOS sets MSL to 15 s; Linux hard-codes `TIME_WAIT` at 60 s.

GRADING Full 20 or nothing. A pass is every round with a prediction and an observation written in, and Round 4 attempted, by hand or with any command line agent. A group on another agent should note how its results differ from Claude's; a difference never costs the pass. Wrong predictions pass.